1. Priority Non-PreEmptive

Input:

```
#include <stdio.h>

struct Process {
   int pid;
   int burst;
   int priority;
   int waiting;
   int turnaround;
   int completion;
};

int main() {
   int n, i, j;
   struct Process p[20], temp;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++) {
      printf("\nProcess %d\n", i + 1);
      p[i].pid = i + 1;

      printf("Enter Burst Time: ");
      scanf("%d", &p[i].burst);

      printf("Enter Priority: ");
      scanf("%d", &p[i].priority);
   }

   // Sort by priority
   for(i = 0; i < n - 1; i++) {
      for(j = i + 1; j < n; j++) {
         if(p[i].priority > p[j].priority) {
            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
         }
      }
   }

   // Waiting time
   p[0].waiting = 0;
   for(i = 1; i < n; i++) {
      p[i].waiting = 0;
      for(j = 0; j < i; j++) {
```

```
            p[i].waiting += p[j].burst;
        }
    }

    // Turnaround time and Completion time
    for(i = 0; i < n; i++) {
        p[i].turnaround = p[i].burst + p[i].waiting;
        p[i].completion = p[i].turnaround;
    }

    float total_wt = 0, total_tat = 0;

    printf("\nPID\tBT\tPR\tWT\tTAT\tCT\n");

    for(i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
              p[i].pid, p[i].burst, p[i].priority,p[i].completion,
              p[i].waiting, p[i].turnaround);

        total_wt += p[i].waiting;
        total_tat += p[i].turnaround;
    }

    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}
```

Output:

```
Enter number of processes: 4

Process 1
Enter Burst Time: 3
Enter Priority: 1

Process 2
Enter Burst Time: 4
Enter Priority: 3

Process 3
Enter Burst Time: 3
Enter Priority: 2

Process 4
Enter Burst Time: 3
Enter Priority: 6

PID     BT      PR      WT      TAT     CT
1       3       1       3       0       3
3       3       2       6       3       6
2       4       3       10      6       10
4       3       6       13      10      13

Average Waiting Time = 4.75
Average Turnaround Time = 8.00
PS C:\Users\Parv Jain\OneDrive\Desktop\OS Lab>
```

2. Priority Pre-Emptive

Input:

```
#include <stdio.h>

int main() {
   int n, i, time = 0, completed = 0;
   int bt[20], at[20], pr[20], rt[20];
   int wt[20], tat[20], ct[20];
   int highest, min_pr;
   int total_wt = 0, total_tat = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++) {
      printf("\nProcess %d\n", i + 1);

      printf("Enter Arrival Time: ");
      scanf("%d", &at[i]);

      printf("Enter Burst Time: ");
      scanf("%d", &bt[i]);

      printf("Enter Priority: ");
      scanf("%d", &pr[i]);

      rt[i] = bt[i];
   }

   while(completed < n) {
      highest = -1;
      min_pr = 9999;

      for(i = 0; i < n; i++) {
         if(at[i] <= time && rt[i] > 0 && pr[i] < min_pr) {
            min_pr = pr[i];
            highest = i;
         }
      }

      if(highest == -1) {
         time++;
         continue;
      }

      rt[highest]--;
```

```
        time++;

        if(rt[highest] == 0) {
            completed++;

            ct[highest] = time;
            tat[highest] = ct[highest] - at[highest];
            wt[highest] = tat[highest] - bt[highest];

            total_wt += wt[highest];
            total_tat += tat[highest];
        }
    }

    printf("\nPID\tAT\tBT\tPR\tCT\tWT\tTAT\n");

    for(i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], pr[i], ct[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", (float)total_wt/n);
    printf("\nAverage Turnaround Time = %.2f\n", (float)total_tat/n);

    return 0;
}
```

Output:

```
Enter number of processes: 4

Process 1
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Priority: 1

Process 2
Enter Arrival Time: 2
Enter Burst Time: 4
Enter Priority: 3

Process 3
Enter Arrival Time: 5
Enter Burst Time: 3
Enter Priority: 5

Process 4
Enter Arrival Time: 6
Enter Burst Time: 3
Enter Priority: 6

PID     AT      BT      PR      CT      WT      TAT
1       0       3       1       3       0       3
2       2       4       3       7       1       5
3       5       3       5       10      2       5
4       6       3       6       13      4       7

Average Waiting Time = 1.75
Average Turnaround Time = 5.00
PS C:\Users\Parv Jain\OneDrive\Desktop\OS Lab>
```

3. Round Robin

Input:

```
#include <stdio.h>

int main() {
    int n, tq;
    int pid[20], at[20], bt[20], rt[20];
    int wt[20] = {0}, tat[20] = {0}, ct[20];
    int queue[100];
    int front = 0, rear = 0;
    int visited[20] = {0};

    int current_time = 0, completed = 0;
    float total_wt = 0, total_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter time quantum: ");
    scanf("%d", &tq);

    for(int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);

        printf("Enter Process ID: ");
        scanf("%d", &pid[i]);

        printf("Enter Arrival Time: ");
        scanf("%d", &at[i]);

        printf("Enter Burst Time: ");
        scanf("%d", &bt[i]);

        rt[i] = bt[i];
    }

    while(completed < n) {

        for(int i = 0; i < n; i++) {
            if(at[i] <= current_time && visited[i] == 0) {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }

        if(front == rear) {
```

```
            current_time++;
            continue;
        }

        int p = queue[front++];

        if(rt[p] > tq) {
            current_time += tq;
            rt[p] -= tq;

            for(int i = 0; i < n; i++) {
                if(at[i] <= current_time && visited[i] == 0) {
                    queue[rear++] = i;
                    visited[i] = 1;
                }
            }

            queue[rear++] = p;
        }
        else {
            current_time += rt[p];

            ct[p] = current_time;
            tat[p] = ct[p] - at[p];
            wt[p] = tat[p] - bt[p];

            rt[p] = 0;
            completed++;
        }
    }

    printf("\nPID\tAT\tBT\tCT\tWT\tTAT\n");

    for(int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i], wt[i], tat[i]);

        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}
```

Output:

```
Enter number of processes: 4
Enter time quantum: 2

Process 1
Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 3

Process 2
Enter Process ID: 2
Enter Arrival Time: 2
Enter Burst Time: 4

Process 3
Enter Process ID: 3
Enter Arrival Time: 5
Enter Burst Time: 4

Process 4
Enter Process ID: 3
Enter Arrival Time: 6
Enter Burst Time: 3

PID     AT      BT      CT      WT      TAT
1       0       3       5       2       5
2       2       4       7       1       5
3       5       4       13      4       8
3       6       3       14      5       8

Average Waiting Time = 3.00
Average Turnaround Time = 6.50
PS C:\Users\Parv Jain\OneDrive\Desktop\OS Lab>
```